

PetCare API

Diagramas de Flujo y Casos de Uso

Versión 1.1 · Flujos operativos corregidos y blueprint de Postman reproducible

INTEGRARTEC · INTEGRADOR 4

Propósito del documento Visualizar cómo interactúan los actores con PetCare API, cómo recorre cada petición las capas de NestJS, qué decisiones de negocio pueden desviar el flujo y cómo se documentarán/prueban esos recorridos mediante Swagger y una colección de Postman.

PetCare V1 · Documento 03 de 03 · Septiembre 2026

Control del documento

Campo	Definición
Proyecto	PetCare API
Documento	03 — Diagramas de Flujo y Casos de Uso
Versión	1.1
Estado	Baseline pre-implementación
Fecha	09/09/2026
Documento relacionado 01	Especificación Funcional de la API (Contrato) v1.2
Documento relacionado 02	Diccionario de Errores y Reglas de Negocio v1.1
Alcance	PetCare API V1 — sin frontend ni integraciones externas de terceros

Cómo debe leerse este documento

El Documento 01 define qué ofrece la API. El Documento 02 define qué reglas y errores gobiernan cada operación. Este Documento 03 muestra cómo esos contratos se encadenan en escenarios reales y establece el guion de pruebas que luego se materializará en Postman cuando la API esté implementada.

- Los diagramas de secuencia representan decisiones lógicas del diseño; no agregan endpoints ni reglas nuevas.
- Cada caso de uso referencia endpoints, roles, reglas BR-* y errores del Documento 02.
- La colección de Postman descrita aquí es un blueprint pre-implementación. Los ejemplos reales de response se capturarán cuando el backend esté ejecutable.
- Swagger y Postman cumplen funciones complementarias: Swagger descubre/prueba el contrato; Postman conserva escenarios reproducibles y secuencias de prueba.

Índice funcional

1. Contexto e integración V1
2. Actores y casos de uso
3. Flujo transversal de una petición
4. Catálogo de casos de uso UC-*
5. Diagramas de secuencia principales
6. Flujos de error y autorización
7. Blueprint de colección Postman
8. Estrategia de pruebas por flujo
9. Trazabilidad con Documentos 01 y 02

10. Criterios de cierre del Documento 03

1. Contexto e integración V1

PetCare V1 es una API backend. No incluye frontend, correo, almacenamiento de archivos, pagos ni integraciones veterinarias. Sus únicos participantes técnicos persistentes son el cliente HTTP, la aplicación NestJS y PostgreSQL mediante Prisma.

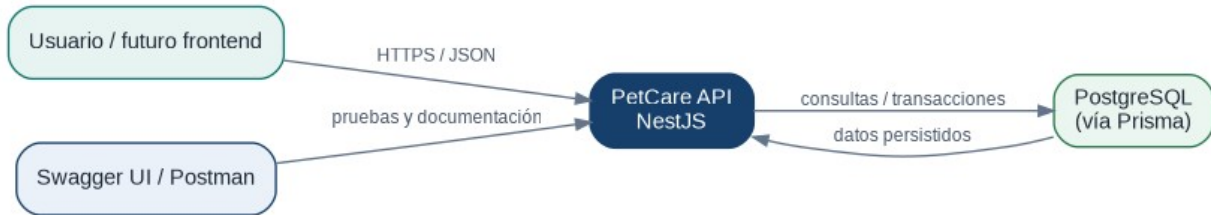


Figura 1 — Contexto técnico de PetCare V1.

Decisión de alcance: no existe un “sistema ONG” externo en V1. Una ONG puede aparecer como texto descriptivo en `rescueOrganizationName` y una persona vinculada a ella accede como usuario PetCare normal con un rol `PetAccess`.

1.1 Clientes previstos

Cliente	Uso en V1	Estado
Swagger UI	Descubrir endpoints, schemas y probar requests manualmente.	Comprometido
Postman	Ejecutar flujos completos, guardar variables y casos de error.	Comprometido como herramienta de verificación
Frontend web/móvil	Consumidor futuro de la API.	Fuera de alcance del Integrador 4

2. Actores y casos de uso

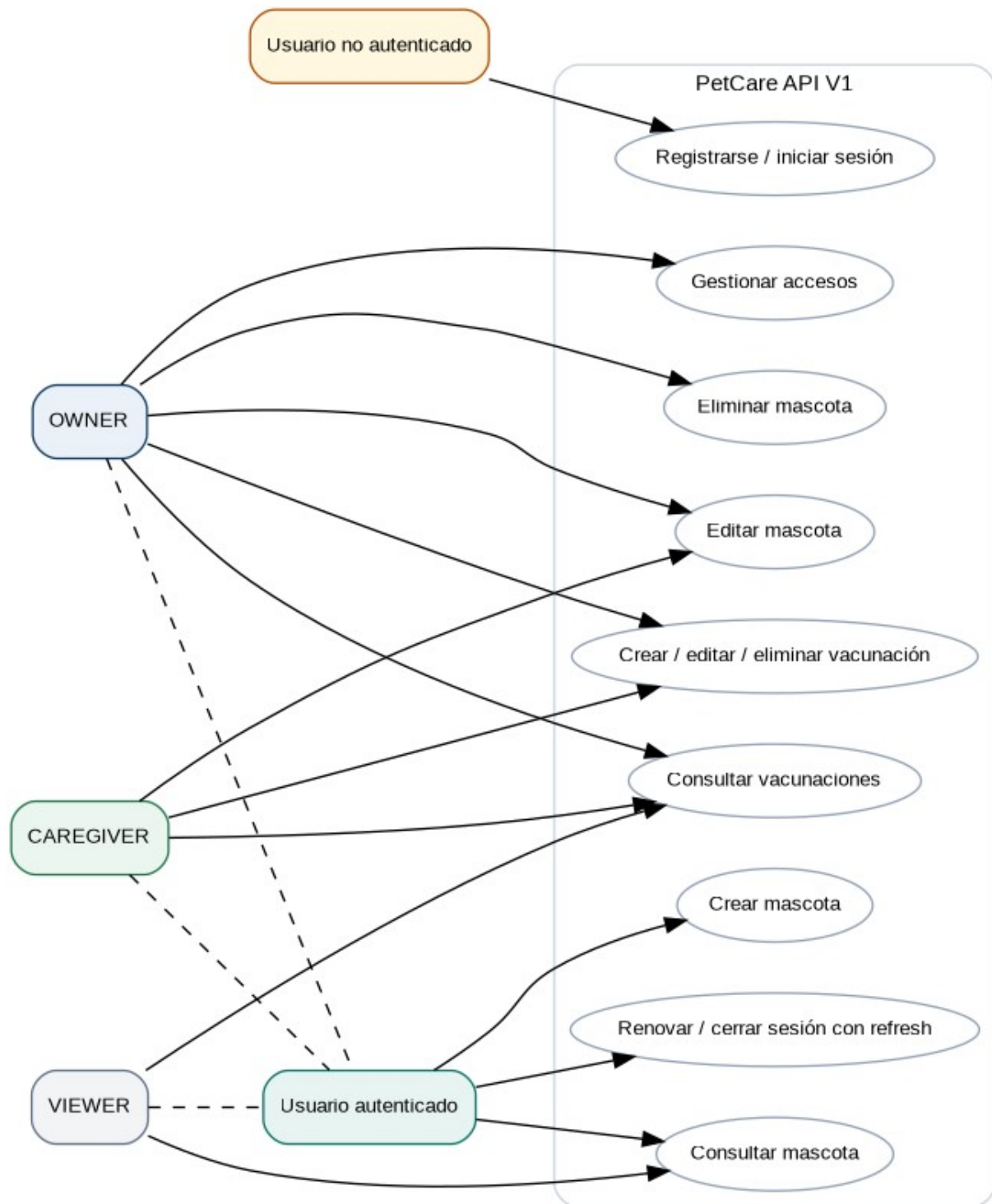


Figura 2 — Casos de uso principales y alcance por rol.

2.1 Actores funcionales

Actor	Descripción	Observación
Usuario no autenticado	Persona sin access token vigente.	Puede health/register/login. Refresh y logout no usan access token, pero exigen un refreshToken válido en el body.
Usuario autenticado	Usuario con access token válido.	Puede crear mascotas y usar recursos según PetAccess.
OWNER	Administrador de una mascota.	Gestiona accesos y puede eliminar la mascota.
CAREGIVER	Responsable activo del cuidado.	Mantiene datos básicos y vacunaciones.
VIEWER	Responsable de consulta.	No realiza escrituras sobre Pet/Vaccination.

2.2 Catálogo resumido

ID	Caso de uso	Actor principal	Resultado esperado
UC-01	Registrarse e iniciar sesión	Usuario no autenticado	Cuenta creada y sesión con access + refresh.
UC-02	Crear mascota	Usuario autenticado	Pet y PetAccess OWNER creados de forma atómica.
UC-03	Consultar mascotas accesibles	OWNER / CAREGIVER / VIEWER	Solo se listan mascotas con PetAccess del usuario.
UC-04	Compartir acceso	OWNER	Usuario existente obtiene OWNER, CAREGIVER o VIEWER.
UC-05	Registrar o mantener vacunación	OWNER / CAREGIVER	Vaccination persistida con fechas válidas.
UC-06	Intentar escritura sin permiso	VIEWER	Operación rechazada con 403 PET_ROLE_FORBIDDEN.
UC-07	Administrar responsables sin perder último OWNER	OWNER	Cambio permitido o 409 PET_LAST_OWNER.
UC-08	Renovar/cerrar sesión	Usuario con refresh válido	Refresh válido renueva access; logout revoca refresh.

ID	Caso de uso	Actor principal	Resultado esperado
UC-09	Eliminar mascota	OWNER	Pet y relaciones dependientes eliminadas.

3. Flujo transversal de una petición

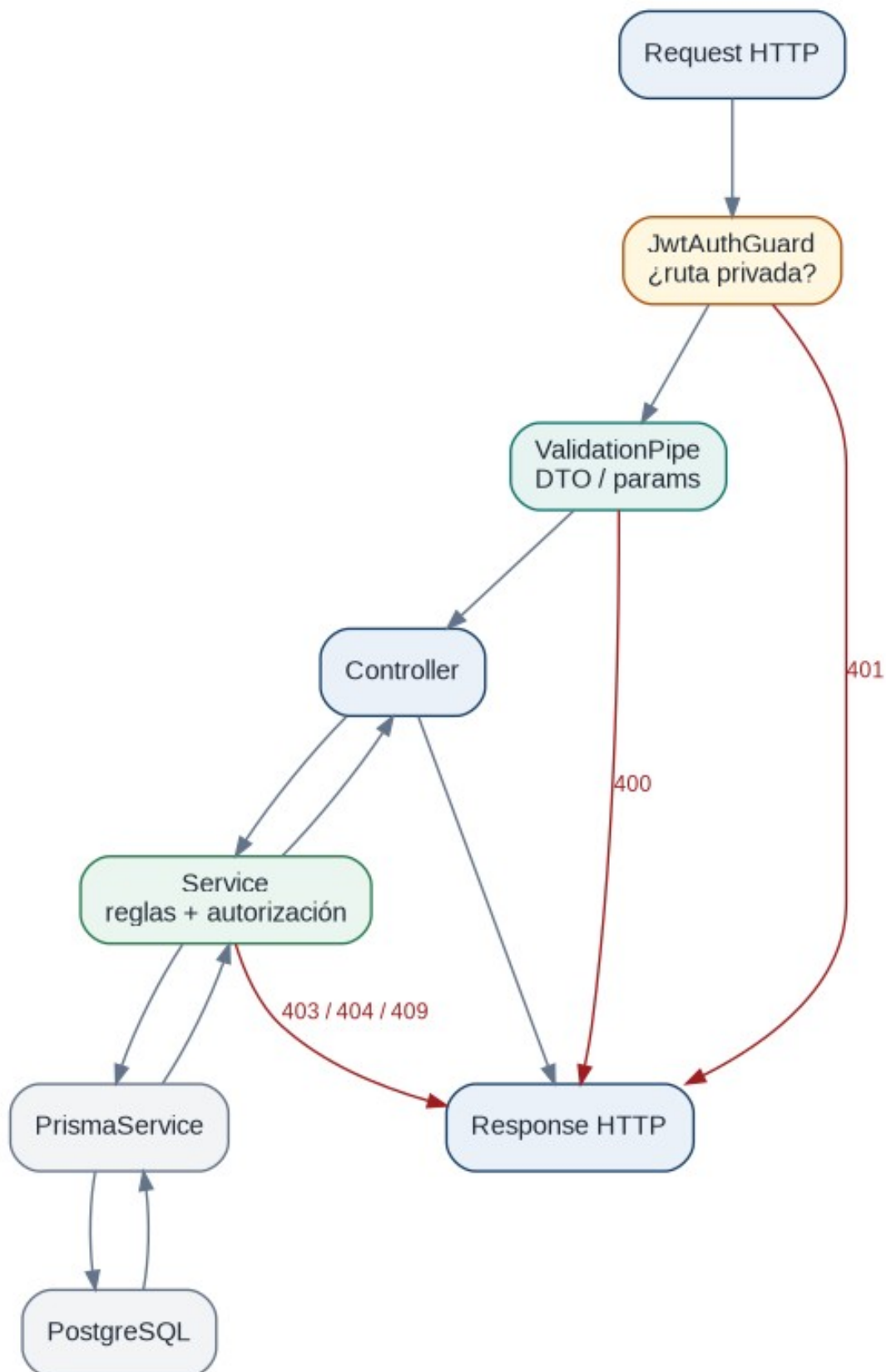


Figura 3 — Recorrido lógico de una petición privada en NestJS.

3.1 Lectura del flujo

1. La request entra a NestJS. Si la ruta es privada, JwtAuthGuard exige un access token válido.
2. ValidationPipe valida DTOs y parámetros. Las propiedades no declaradas se rechazan.
3. El Controller traduce HTTP a una llamada de aplicación y delega la lógica.
4. El Service aplica reglas de negocio y autorización sobre PetAccess.
5. PrismaService consulta o modifica PostgreSQL, usando transacciones cuando la operación debe ser atómica.
6. NestJS devuelve la respuesta exitosa o una excepción normalizada según el Documento 02.

401, 400, 403/404/409 no representan el mismo tipo de fallo. El orden de evaluación importa: una request sin JWT debe fallar por autenticación antes de evaluar el rol sobre una mascota.

4. Catálogo de casos de uso UC-*

UC-01 — Registrarse e iniciar sesión

Campo	Definición
Actor principal	Usuario no autenticado
Precondiciones	No requiere sesión. Para login debe existir una cuenta.
Disparador	El usuario desea comenzar a usar PetCare.
Postcondición	Existe un usuario autenticable y una sesión refresh activa.
Endpoints	POST /auth/register · POST /auth/login
Trazabilidad	BR-AUTH-001..006 · AC-01..03

Flujo principal

1. Envía RegisterRequest a POST /auth/register.
2. La API valida DTO, normaliza email y comprueba unicidad.
3. AuthService hashea la contraseña y persiste el usuario.
4. El usuario envía email/password a POST /auth/login.
5. La API valida credenciales, genera access + refresh y persiste el hash del refresh.
6. El cliente recibe AuthResponse.

Flujos alternativos / fallidos

- Email ya registrado → 409 AUTH_EMAIL_IN_USE.
- Credenciales incorrectas → 401 AUTH_INVALID_CREDENTIALS.
- Demasiados intentos → 429 RATE_LIMIT_EXCEEDED.

UC-02 — Crear mascota

Campo	Definición
Actor principal	Usuario autenticado
Precondiciones	Access token válido.
Disparador	El usuario registra una mascota bajo su responsabilidad.
Postcondición	Pet existe y conserva al menos un OWNER.
Endpoints	POST /pets
Trazabilidad	BR-PET-001, BR-PET-005 · AC-05

Flujo principal

1. Envía CreatePetRequest a POST /pets.
2. La API valida campos y birthDate.
3. PetsService inicia una operación atómica.
4. Se crea Pet.
5. Se crea PetAccess del creador con role=OWNER.
6. La transacción confirma ambas escrituras y devuelve PetResponse con myRole=OWNER.

Flujos alternativos / fallidos

- birthDate futura → 400 PET_INVALID_BIRTH_DATE.
- Si falla Pet o PetAccess → rollback; no queda una mascota sin responsable.

UC-03 — Consultar mascotas accesibles

Campo	Definición
Actor principal	OWNER / CAREGIVER / VIEWER
Precondiciones	Access token válido.
Disparador	El usuario abre su listado de mascotas.
Postcondición	El usuario obtiene únicamente información autorizada.
Endpoints	GET /pets · GET /pets/:petId
Trazabilidad	BR-PET-002, BR-PET-003 · AC-06

Flujo principal

1. Envía GET /pets.
2. El backend identifica al usuario desde JWT.
3. Prisma consulta Pet relacionados mediante PetAccess del usuario.

4. La API devuelve solo el conjunto accesible, incluyendo myRole.

Flujos alternativos / fallidos

- Sin access token válido → 401 AUTH_ACCESS_REQUIRED.
- El listado puede ser vacío; no es un error.

UC-04 — Compartir acceso

Campo	Definición
Actor principal	OWNER
Precondiciones	El solicitante es OWNER; el destinatario ya tiene cuenta PetCare.
Disparador	El OWNER quiere sumar a otra persona al cuidado.
Postcondición	El usuario destino puede acceder a la mascota según su rol.
Endpoints	POST /pets/:petId/access
Trazabilidad	BR-ACCESS-001..005 · AC-07..09

Flujo principal

1. Envía email y role a POST /pets/:petId/access.
2. La API verifica PetAccess del solicitante y exige OWNER.
3. Busca al usuario destino por email normalizado.
4. Comprueba que no exista ya userId+petId.
5. Crea PetAccess con el rol solicitado y devuelve 201.

Flujos alternativos / fallidos

- Usuario destino inexistente → 404 USER_NOT_FOUND.
- Acceso ya existente → 409 PET_ACCESS_EXISTS.
- CAREGIVER/VIEWER intenta administrar accesos → 403 PET_ROLE_FORBIDDEN.

UC-05 — Registrar o mantener vacunación

Campo	Definición
Actor principal	OWNER / CAREGIVER
Precondiciones	PetAccess con rol de escritura.
Disparador	El responsable registra una vacuna aplicada o actualiza sus datos.
Postcondición	La vacunación queda persistida y consultable por responsables con acceso.
Endpoints	POST/PATCH/DELETE /pets/:petId/vaccinations/...

Campo	Definición
Trazabilidad	BR-VACC-001..005 · AC-11

Flujo principal

1. Envía CreateVaccinationRequest a POST /pets/:petId/vaccinations.
2. La API confirma que la mascota existe y que el usuario puede escribir.
3. Valida appliedAt <= hoy y nextDueAt >= appliedAt cuando existe próxima fecha.
4. Persiste Vaccination asociada al petId.
5. Devuelve 201 y el registro creado.

Flujos alternativos / fallidos

- VIEWER → 403 PET_ROLE_FORBIDDEN.
- appliedAt futura o nextDueAt anterior → 400 VACCINATION_INVALID_DATES.
- En PATCH, las fechas se validan sobre el estado final combinado con los valores persistidos.
- Pet inexistente → 404 PET_NOT_FOUND.

UC-06 — Intentar escritura sin permiso

Campo	Definición
Actor principal	VIEWER
Precondiciones	JWT válido y PetAccess VIEWER.
Disparador	El VIEWER intenta modificar Pet o Vaccination.
Postcondición	No se modifica ningún dato.
Endpoints	PATCH /pets/:petId · POST/PATCH/DELETE vaccinations
Trazabilidad	BR-PET-004 · BR-VACC-002 · AC-09

Flujo principal

1. La request supera autenticación porque el JWT es válido.
2. El Service carga PetAccess.
3. Detecta role=VIEWER para una acción de escritura.
4. Lanza PET_ROLE_FORBIDDEN y NestJS responde 403.

Flujos alternativos / fallidos

- Si no hubiera JWT, el flujo terminaría antes con 401 AUTH_ACCESS_REQUIRED.
- Si no existiera PetAccess, el error sería 403 PET_ACCESS_DENIED.

UC-07 — Administrar responsables sin perder último OWNER

Campo	Definición
Actor principal	OWNER

Campo	Definición
Precondiciones	Mascota existente y solicitante OWNER.
Disparador	Se quiere cambiar rol o quitar a un responsable.
Postcondición	La mascota conserva como mínimo un OWNER.
Endpoints	PATCH/DELETE /pets/:petId/access/:userId
Trazabilidad	BR-PET-005 · BR-ACCESS-003..005 · AC-07

Flujo principal

1. La API carga el acceso objetivo y cantidad de OWNER actuales.
2. Si el acceso objetivo no es el último OWNER, prepara PATCH/DELETE.
3. La comprobación y la modificación se ejecutan en una transacción serializable; ante conflicto concurrente la operación se reintenta o falla sin dejar cero OWNER.
4. La modificación confirmada devuelve 200/204.

Flujos alternativos / fallidos

- Quitar o degradar al único OWNER → 409 PET_LAST_OWNER.
- Acceso objetivo inexistente → 404 PET_ACCESS_NOT_FOUND.
- Rol solicitado inválido → 400 VALIDATION_ERROR.

UC-08 — Renovar y cerrar sesión

Campo	Definición
Actor principal	Usuario con refresh de sesión
Precondiciones	refreshToken válido para renovar o cerrar sesión. No se requiere access token en estos dos endpoints.
Disparador	El access expiró o el usuario decide cerrar sesión.
Postcondición	La sesión puede continuar con un nuevo access o quedar sin capacidad de renovación.
Endpoints	POST /auth/refresh · POST /auth/logout
Trazabilidad	BR-AUTH-005..007 · AC-03

Flujo principal

1. POST /auth/refresh recibe { refreshToken } y valida firma, expiración y correspondencia con refreshTokenHash.
2. Si coincide, genera solamente un nuevo accessToken; el refresh no rota en V1.
3. POST /auth/logout recibe { refreshToken }, vuelve a validarlo y deja refreshTokenHash en null.
4. Luego del logout ese refresh ya no puede renovar la sesión y el cliente descarta sus tokens.

Flujos alternativos / fallidos

- Refresh anterior reemplazado por un nuevo login → 401 AUTH_REFRESH_INVALID.
- Refresh expirado/revocado → 401 AUTH_REFRESH_INVALID.
- Un access JWT ya emitido puede seguir siendo válido hasta su expiración corta; V1 no mantiene blacklist.

UC-09 — Eliminar mascota

Campo	Definición
Actor principal	OWNER
Precondiciones	Usuario autenticado y OWNER.
Disparador	El OWNER decide eliminar definitivamente el registro.
Postcondición	No quedan datos dependientes huérfanos.
Endpoints	DELETE /pets/:petId
Trazabilidad	BR-PET-006 · AC-10

Flujo principal

1. Envía DELETE /pets/:petId.
2. La API verifica acceso y rol OWNER.
3. Prisma elimina Pet y relaciones dependientes PetAccess/Vaccination según el diseño de cascada.
4. La API responde 204 No Content.

Flujos alternativos / fallidos

- CAREGIVER/VIEWER → 403 PET_ROLE_FORBIDDEN.
- Mascota inexistente → 404 PET_NOT_FOUND.

5. Diagramas de secuencia principales

Los siguientes diagramas muestran la colaboración entre cliente, Controller, Service, Prisma y PostgreSQL. Simplifican detalles internos para destacar las decisiones que deben poder defenderse durante el examen oral.

5.1 UC-01 — Registro e inicio de sesión

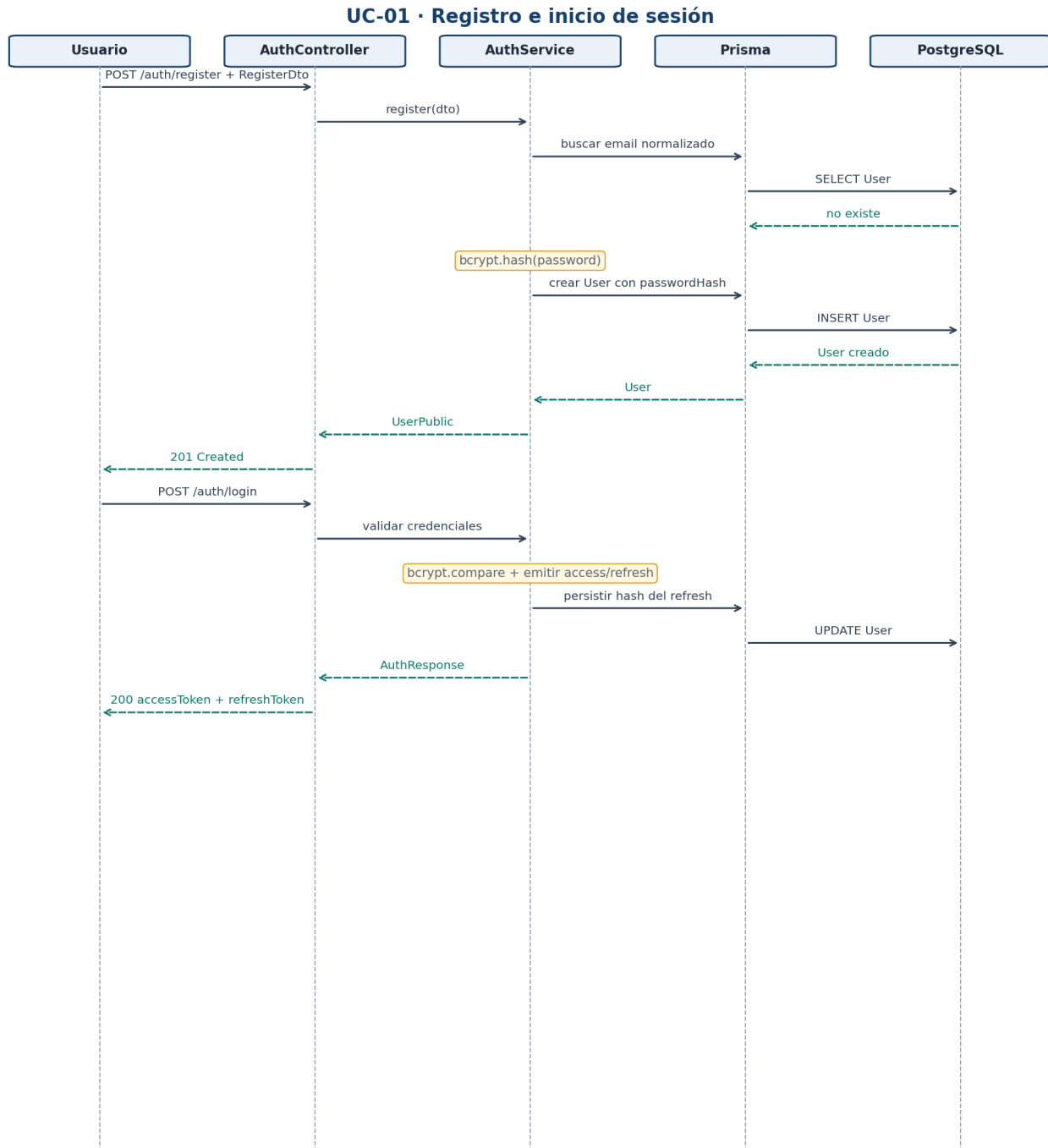


Figura 4 — Registro y login: validación, bcrypt, JWT y persistencia del hash de refresh.

5.2 UC-02 — Crear mascota

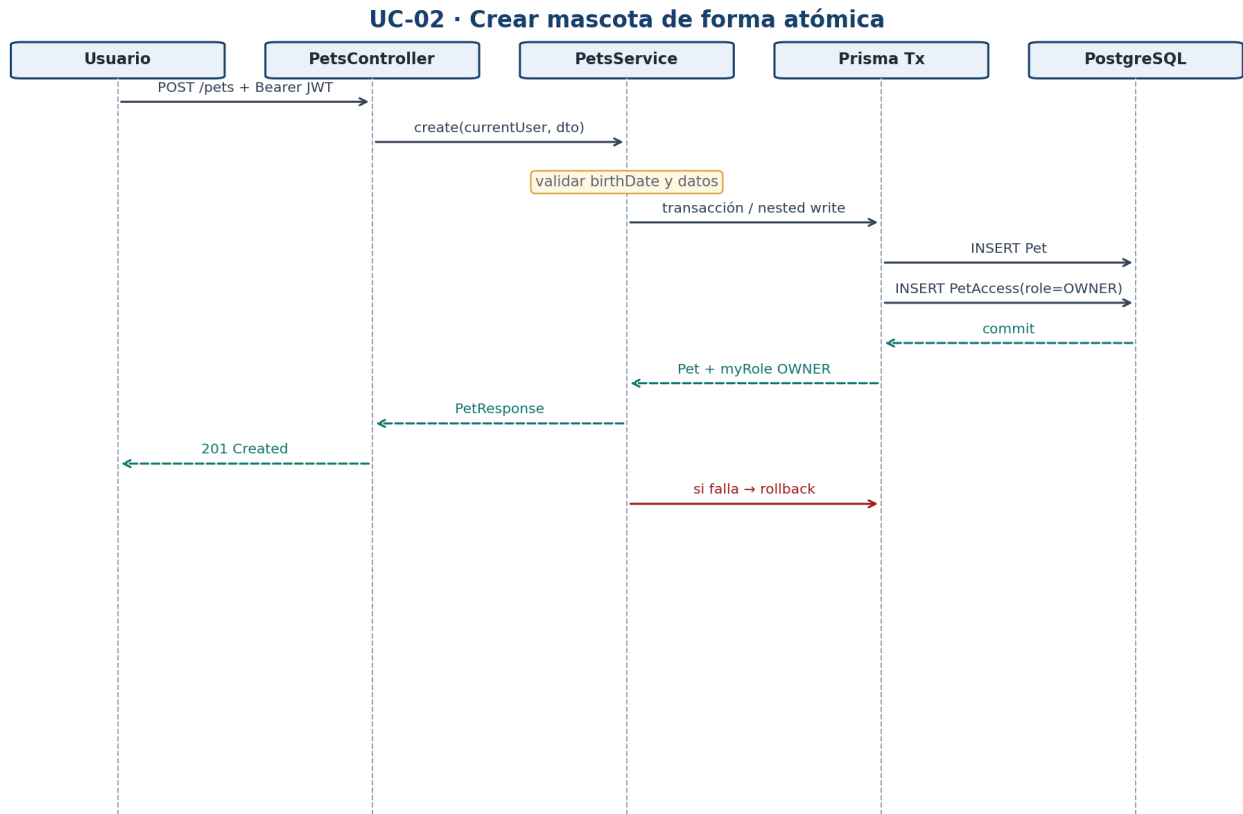


Figura 5 — Pet y PetAccess OWNER forman una única operación de negocio.

5.3 UC-04 — Compartir acceso

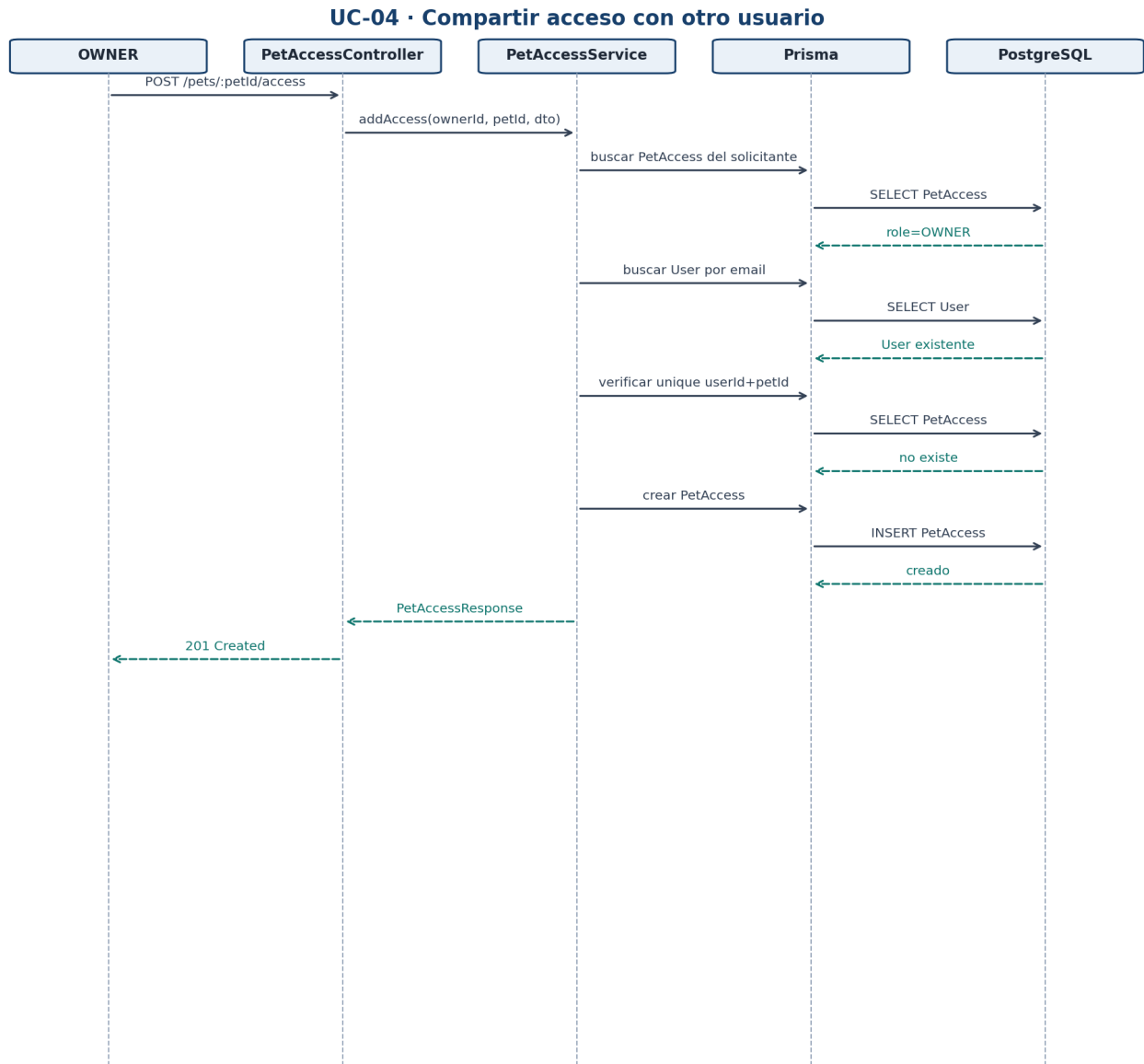


Figura 6 — OWNER agrega un usuario ya registrado y evita accesos duplicados.

5.4 UC-05 — Registrar vacunación

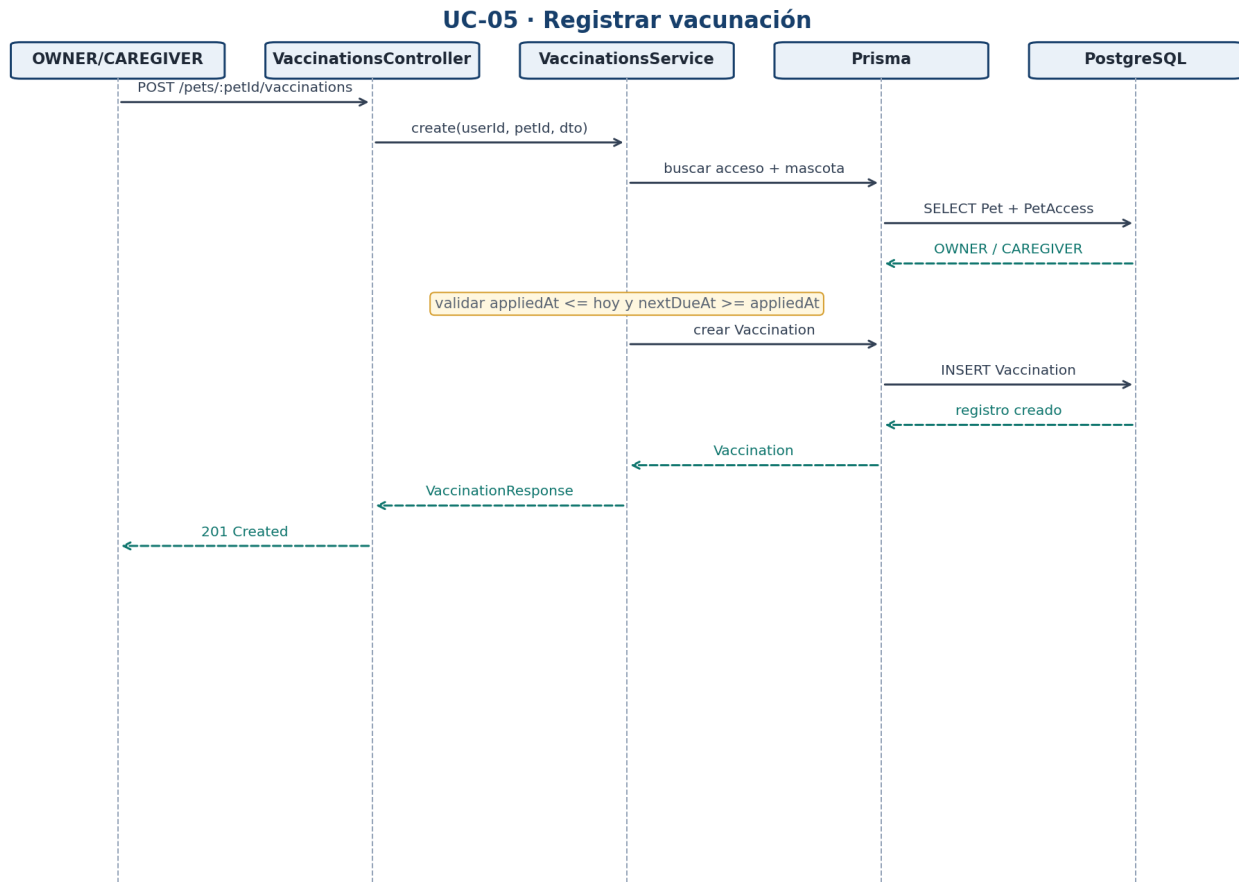


Figura 7 — Autorización por PetAccess antes de persistir Vaccination.

5.5 UC-06 — VIEWER intenta escribir

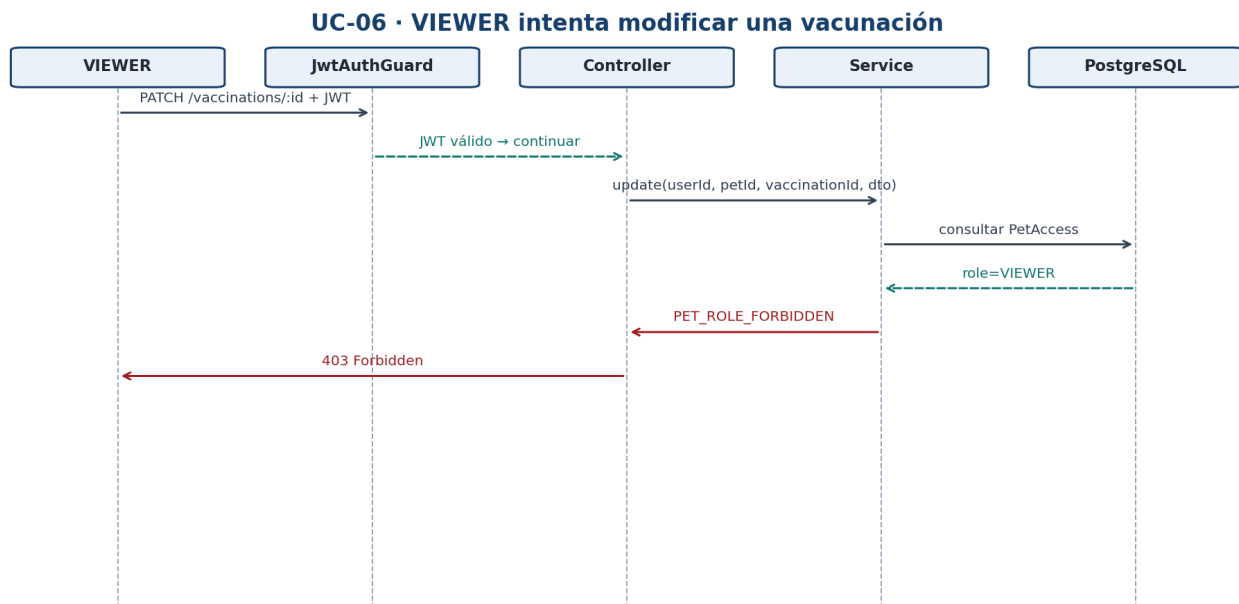


Figura 8 — JWT válido no implica autorización suficiente: resultado 403.

5.6 UC-07 — Último OWNER

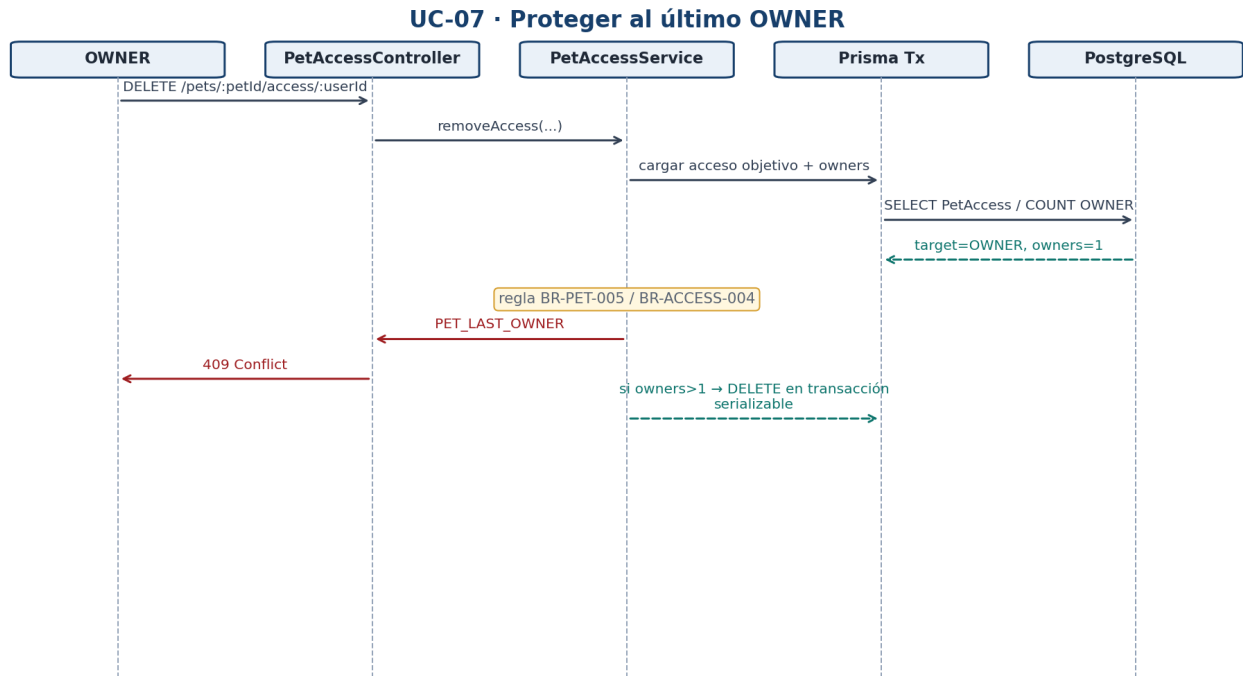


Figura 9 — Regla de negocio que impide dejar una mascota sin administrador.

5.7 UC-08 — Refresh y logout

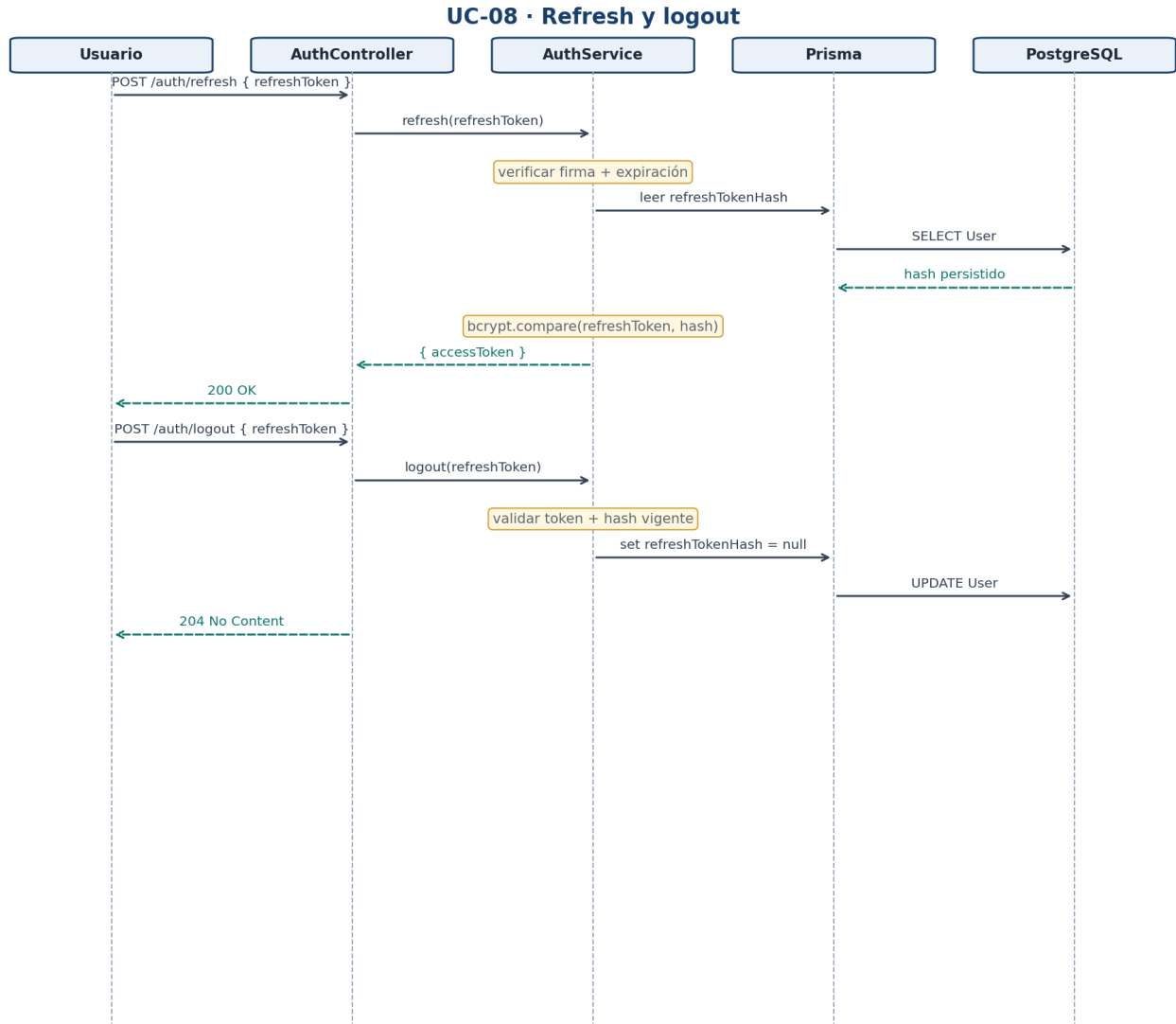


Figura 10 — Renovación de access y revocación del refresh persistente.

6. Flujos de error y autorización

6.1 Árbol mínimo de decisión para una ruta privada

```

¿Access token válido?
├ NO → 401 AUTH_ACCESS_REQUIRED
└ SÍ
  ↓
¿petId / vaccinationId tienen formato válido?
├ NO → 400 INVALID_IDENTIFIER
└ SÍ
  ↓
¿Existe el recurso?
├ NO → 404 *_NOT_FOUND
└ SÍ
  ↓
¿El usuario tiene PetAccess?
├ NO → 403 PET_ACCESS_DENIED
└ SÍ
  ↓
¿Su rol permite la acción?
├ NO → 403 PET_ROLE_FORBIDDEN
└ SÍ
  ↓
¿Se cumplen reglas específicas?
├ NO → 400 / 409 según Documento 02
└ SÍ → ejecutar operación
  
```

6.2 Diferencia operativa 401 vs 403

Situación	Identidad conocida	PetAccess	Respuesta
No envía JWT	No	No evaluado	401 AUTH_ACCESS_REQUIRED
JWT inválido/expirado	No confiable	No evaluado	401 AUTH_ACCESS_REQUIRED
JWT válido, sin relación con Pet	Sí	No	403 PET_ACCESS_DENIED
JWT válido + VIEWER intenta escribir	Sí	Sí, VIEWER	403 PET_ROLE_FORBIDDEN
JWT válido + OWNER	Sí	Sí, OWNER	Continúa

6.3 Conflictos 409 del MVP

Código	Caso	Por qué es 409 y no 400
AUTH_EMAIL_IN_USE	El email ya pertenece a otra cuenta.	El request es válido, pero entra en conflicto con UNIQUE existente.
PET_ACCESS_EXISTS	Ese usuario ya tiene PetAccess.	La relación solicitada contradice el estado actual.

Código	Caso	Por qué es 409 y no 400
PET_LAST_OWNER	Se quitaría/degradaría al único OWNER.	Los datos son válidos, pero violan una invariante del negocio.

7. Blueprint de colección Postman

La colección Postman será el guion reproducible de demostración y QA. Como el backend todavía no está implementado, este documento define la estructura, variables y asserts. Los responses reales se capturarán cuando existan la API local y el deploy.

7.1 Estructura de carpetas

```

PetCare API V1
├── 00 - Health
│   └── Health check
├── 01 - Auth / Setup
│   ├── Generate runId
│   ├── Register OWNER / CAREGIVER / VIEWER / OUTSIDER
│   ├── Login OWNER / CAREGIVER / VIEWER / OUTSIDER
│   ├── Me
│   ├── Refresh OWNER
│   └── Logout OWNER
├── 02 - Pets
│   ├── Create Pet
│   ├── List Pets
│   ├── Get Pet
│   ├── Update Pet
│   └── Delete Pet
├── 03 - Pet Access
│   ├── List Access
│   ├── Add Caregiver
│   ├── Change Role
│   └── Remove Access
├── 04 - Vaccinations
│   ├── Create Vaccination
│   ├── List Vaccinations
│   ├── Get Vaccination
│   ├── Update Vaccination
│   └── Delete Vaccination
└── 05 - Error Scenarios
    ├── Invalid DTO → 400
    ├── No JWT → 401
    ├── Viewer writes → 403
    ├── Pet not found → 404
    ├── Duplicate access → 409
    ├── Remove last OWNER → 409
    └── Invalid vaccination dates → 400

```

7.2 Variables de environment

Variable	Ejemplo local	Origen / uso
baseUrl	http://localhost:3000/api/v1	Prefijo común de requests.
runId	<runtime>	Timestamp/UUID generado al inicio para que cada corrida use emails únicos.

Variable	Ejemplo local	Origen / uso
ownerEmail	owner+{{runId}}@example.com	Cuenta OWNER de la corrida.
caregiverEmail	caregiver+{{runId}}@example.com	Cuenta CAREGIVER de la corrida.
viewerEmail	viewer+{{runId}}@example.com	Cuenta VIEWER de la corrida.
outsiderEmail	outsider+{{runId}}@example.com	Cuenta autenticada sin PetAccess para pruebas 403.
ownerAccessToken	<runtime>	Login OWNER / refresh.
ownerRefreshToken	<runtime>	Login OWNER; body de refresh/logout.
caregiverAccessToken	<runtime>	Login CAREGIVER.
viewerAccessToken	<runtime>	Login VIEWER.
outsiderAccessToken	<runtime>	Login OUTSIDER.
petId	<runtime>	Se guarda luego de Create Pet.
vaccinationId	<runtime>	Se guarda luego de Create Vaccination.
sharedUserId	<runtime>	Se obtiene al listar accesos.

7.3 Automatización mínima en Postman

```
// Pre-request de la primera request de setup
pm.environment.set("runId", Date.now().toString());

// Ejemplo después de Login OWNER
pm.test("status 200", () => pm.response.to.have.status(200));
const body = pm.response.json();
pm.environment.set("ownerAccessToken", body.accessToken);
pm.environment.set("ownerRefreshToken", body.refreshToken);

// Ejemplo después de Create Pet
pm.test("status 201", () => pm.response.to.have.status(201));
const pet = pm.response.json();
pm.environment.set("petId", pet.id);
pm.expect(pet.myRole).to.eql("OWNER");
```

7.4 Headers y autenticación

- Requests privadas usan el token del actor correspondiente: Bearer {{ownerAccessToken}}, {{caregiverAccessToken}}, {{viewerAccessToken}} u {{outsiderAccessToken}}.
- Content-Type: application/json para requests con body.
- POST /auth/refresh y POST /auth/logout reciben { "refreshToken": "{{ownerRefreshToken}}" } en JSON; no usan access token.
- Los secrets reales nunca se guardan en la colección versionada.

La colección Postman no reemplaza tests automatizados. Sirve como evidencia manual/repetible, smoke test y soporte para la demostración del oral.

8. Estrategia de pruebas por flujo

8.1 Happy path de demostración

Paso	Request	Assert principal	Variable guardada
1	GET /health + generar runId	200	runId
2	Register OWNER / CAREGIVER / VIEWER / OUTSIDER	201 cada uno	emails dinámicos
3	Login de los 4 usuarios	200 + tokens	tokens por actor
4	OWNER → POST /pets	201 + myRole OWNER	petId
5	OWNER → agrega CAREGIVER y VIEWER	201 cada uno	—
6	CAREGIVER → POST vaccination	201	vaccinationId
7	VIEWER → GET vaccination	200	—
8	VIEWER → PATCH vaccination	403 PET_ROLE_FORBIDDEN	—
9	OUTSIDER → GET pet	403 PET_ACCESS_DENIED	—
10	OWNER → POST /auth/refresh con ownerRefreshToken	200	nuevo ownerAccessToken
11	OWNER → POST /auth/logout con ownerRefreshToken	204	—
12	Reusar ownerRefreshToken luego de logout	401 AUTH_REFRESH_INVALID	—

8.2 Escenarios fallidos prioritarios

ID	Escenario	Esperado	Doc. 02
PF-01	Register con propiedad desconocida	400 VALIDATION_ERROR	T-ERR-11
PF-02	Login incorrecto	401 AUTH_INVALID_CREDENTIALS	T-ERR-03

ID	Escenario	Esperado	Doc. 02
PF-03	GET /pets sin JWT	401 AUTH_ACCESS_REQUIRED	T-ERR-04
PF-04	Usuario sin PetAccess consulta Pet	403 PET_ACCESS_DENIED	T-ERR-05
PF-05	VIEWER intenta editar Pet	403 PET_ROLE_FORBIDDEN	T-ERR-06
PF-06	Acceso duplicado	409 PET_ACCESS_EXISTS	T-ERR-07
PF-07	Quitar último OWNER	409 PET_LAST_OWNER	T-ERR-08
PF-08	nextDueAt < appliedAt	400 VACCINATION_INVALID_DATES	T-ERR-09
PF-08B	appliedAt futura	400 VACCINATION_INVALID_DATES	T-ERR-09B
PF-08C	PATCH deja combinación final de fechas inválida	400 VACCINATION_INVALID_DATES	T-ERR-09C
PF-09	vaccinationId de otra mascota	404 VACCINATION_NOT_FOUND	T-ERR-10
PF-10	Refresh anterior después de nuevo login	401 AUTH_REFRESH_INVALID	T-ERR-13

8.3 Qué debe verificarse en PostgreSQL

- passwordHash existe; jamás password en texto plano.
- refreshTokenHash cambia con login y queda null al logout; /refresh no lo rota.
- Crear Pet genera exactamente un PetAccess OWNER para el creador.
- La combinación userId + petId no se duplica.
- Una operación rechazada por permisos no modifica filas.
- Vaccination siempre conserva FK válida hacia Pet.
- Eliminar Pet no deja PetAccess/Vaccination huérfanos.

9. Trazabilidad con Documentos 01 y 02

Caso	Endpoints Doc. 01	Reglas / errores Doc. 02	Criterios AC
UC-01	register / login	BR-AUTH-* · AUTH_*	AC-01, AC-02
UC-02	POST /pets	BR-PET-001/005 · PET_INVALID_BIRTH_DATE	AC-05, AC-10

Caso	Endpoints Doc. 01	Reglas / errores Doc. 02	Criterios AC
UC-03	GET /pets/:petId	BR-PET-002/003 · PET_ACCESS_DENIED	AC-06
UC-04	POST /pets/:id/access	BR-ACCESS-* · USER_NOT_FOUND / PET_ACCESS_EXISTS	AC-07..09
UC-05	Vaccinations CRUD	BR-VACC-* · VACCINATION_*	AC-11
UC-06	PATCH/POST privados	BR-PET-004 / BR-VACC-002 · PET_ROLE_FORBIDDEN	AC-09
UC-07	PATCH/DELETE access	BR-PET-005 · BR-ACCESS-004 · PET_LAST_OWNER	AC-07
UC-08	refresh / logout	BR-AUTH-005..007 · AUTH_REFRESH_INVALID	AC-03
UC-09	DELETE /pets/:id	BR-PET-006	AC-10

9.1 Alineación con el Integrador 4

Los flujos anteriores evidencian los puntos que la consigna pide poder implementar y defender: autenticación access/refresh/logout, bcrypt, rutas protegidas, dos CRUD persistidos, validación de DTOs, modelado relacional y manejo consistente de errores. El frontend queda fuera del alcance; Swagger/Postman sirven para demostrar la API directamente.

El objetivo del Documento 03 no es aumentar el alcance funcional. Su función es convertir el contrato de PetCare en recorridos observables y comprobables.

10. Criterios de cierre del Documento 03

ID	Criterio
D3-01	Todos los endpoints del Documento 01 aparecen en al menos un caso o grupo de pruebas.
D3-02	Los flujos fallidos usan códigos y reglas existentes en el Documento 02; no inventan errores nuevos.
D3-03	Crear Pet muestra explícitamente la atomicidad Pet + PetAccess OWNER.
D3-04	Los diagramas distinguen autenticación (401) de autorización (403).

ID	Criterio
D3-05	Los roles OWNER / CAREGIVER / VIEWER se reflejan de manera consistente.
D3-06	La colección Postman contempla happy path y errores prioritarios.
D3-07	Las variables de Postman permiten ejecutar el flujo sin copiar IDs manualmente.
D3-08	Cuando la API esté implementada, los ejemplos reales de request/response deben sustituir cualquier placeholder.
D3-09	Swagger, Postman, Documento 01 y Documento 02 deben permanecer sincronizados antes del deploy final.

Decisiones congeladas para implementación

- Casos de uso V1: UC-01 a UC-09. No se agregan nuevos módulos para satisfacer diagramas.
- No hay integraciones externas de terceros en V1.
- PetAccess es la fuente de autorización por mascota.
- OWNER, CAREGIVER y VIEWER son los únicos roles técnicos.
- La creación de Pet + OWNER debe ser atómica.
- El último OWNER no puede eliminarse ni degradarse; las operaciones que reducen OWNER usan transacción serializable.
- Refresh/logout reciben refreshToken en JSON, no rotan refresh y logout no blacklistea access JWT.
- Postman usa usuarios y tokens separados por rol, más runId para hacer repetibles las corridas sin chocar con emails UNIQUE.
- Postman se implementa como colección de verificación; no sustituye tests automatizados.
- Cualquier cambio de flujo que altere contrato o errores obliga a actualizar Documentos 01/02 primero.